

Advanced FFT Algorithms and Applications

Signals and Linear Systems

Ashrafur Rahman
Lecturer, CSE, BUET

ashrafur@cse.buet.ac.bd

- **Applications of FFT:** Fast Convolution
 - Circular Convolution
 - Linear Convolution via Zero-Padding
 - Fast Convolution Complexity
- **FFT for Prime N :** Bluestein's Algorithm
 - Chirp-Z Transform Derivation
 - Bluestein's Convolution Interpretation

Applications of FFT

- One of the most fundamental operations in signal processing is convolution.
- **Linear systems are defined by convolutions.** Filtering, reverberation, correlation, and system analysis all rely on standard linear convolution:

$$y[n] = (h * x)[n] = \sum_{m=-\infty}^{\infty} h[m]x[n - m]$$

- Directly computing this takes $O(L \cdot M)$ time, where L and M are lengths of the signals h and x . For long signals (e.g., room impulse responses or audio tracks), this is computationally prohibitive!
- Fortunately, we have the **Convolution Theorem**: Convolution in the time domain is multiplication in the frequency domain.

Circular Convolution: Derivation

- What happens when we multiply two DFTs and take the inverse?
- Let $Y[k] = X_1[k] \cdot X_2[k]$. Then:

$$\begin{aligned}y[n] &= \text{IDFT}\{Y[k]\} = \frac{1}{N} \sum_{k=0}^{N-1} X_1[k] X_2[k] W_N^{-kn} \\&= \frac{1}{N} \sum_{k=0}^{N-1} \left(\sum_{m=0}^{N-1} x_1[m] W_N^{km} \right) X_2[k] W_N^{-kn} \\&= \sum_{m=0}^{N-1} x_1[m] \left(\frac{1}{N} \sum_{k=0}^{N-1} X_2[k] W_N^{-k(n-m)} \right) \\&= \sum_{m=0}^{N-1} x_1[m] \underbrace{\left(\frac{1}{N} \sum_{k=0}^{N-1} X_2[k] W_N^{-k[(n-m) \bmod N]} \right)}_{\text{IDFT of } X_2 \text{ at index } (n-m) \bmod N} \\&= \sum_{m=0}^{N-1} x_1[m] x_2[(n-m) \bmod N]\end{aligned}$$

- Noticing that $W_N^{-k(n-m+tN)} = W_N^{-k(n-m)} \underbrace{W_N^{-ktN}}_{=e^{-j2\pi kt}=1} = W_N^{-k(n-m)}$ for any integer t .

Circular Convolution Theorem

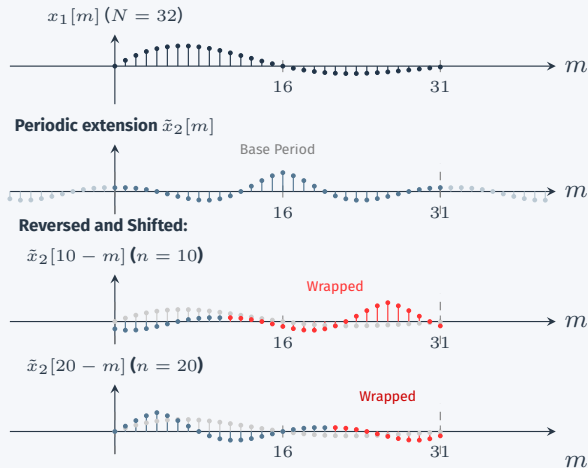
DFT multiplication corresponds to **circular** (not linear) convolution:

$$\text{IDFT}\{X_1[k] \cdot X_2[k]\} = (x_1 \circledast x_2)[n] = \sum_{m=0}^{N-1} x_1[m] x_2[(n - m) \bmod N]$$

- Circular convolution is a special case of linear convolution where the signals are periodic.
- Can we use circular convolution to compute the more useful linear convolution?
- Let's first see an example of circular convolution.

Visualizing Circular Convolution ($N = 32$)

- Circular convolution shifts a **periodic extension** of x_2 , denoted $\tilde{x}_2[m]$.
- Let x_1 and x_2 have $N = 32$ samples.
- To compute $y_c[n] = \sum_{m=0}^{31} x_1[m] \tilde{x}_2[n - m]$:
 1. Extend x_2 periodically.
 2. Time-reverse and shift by n to get $\tilde{x}_2[n - m]$.
 3. Multiply element-wise with $x_1[m]$ (for $m = 0 \dots 31$) and sum.
- The **red** stems highlight values that “wrapped around” from the end of the base sequence.
- In **linear convolution**, we do not want this **wrap**-around effect.



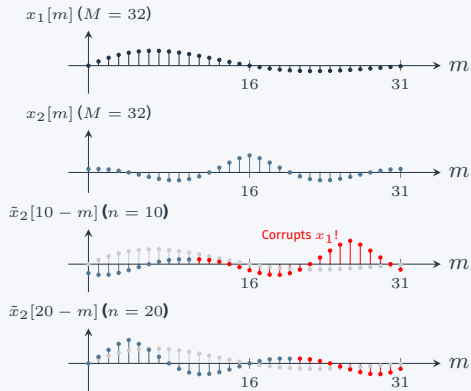
- To use the FFT for standard **Linear Convolution** without introducing wrap-around artifacts from circularity, we must **zero-pad**!
- If $x_1[n]$ has length L and $x_2[n]$ has length M , their linear convolution has length $L + M - 1$.
- We append zeros to both sequences until they share a length $N \geq L + M - 1$. Now, applying circular convolution safely mirrors linear convolution because the wrap-around hits entirely zeros.

Fast Convolution

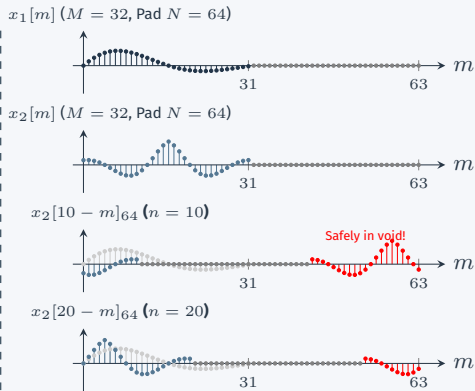
1. Choose $N \geq L + M - 1$, ideally a power of 2 for maximum FFT performance.
2. Zero-pad x_1 and x_2 to length N .
3. Compute $X[1] = \text{FFT}(x_1, N)$ and $X[2] = \text{FFT}(x_2, N)$.
4. Multiply Element-wise: $Y[k] = X[1][k]X[2][k]$.
5. Compute $y[n] = \text{IFFT}(Y, N)$.

Circular vs Linear Convolution: The Zero-Padding Fix

Circular ($N = 32$)



Zero-Padded Linear ($N = 64$)



- **Left:** $N = 32 < L + M - 1 = 63$ — convolution tail wraps around, corrupting early x_1 samples!
- **Right:** $N = 64 \geq 63$ — wrapped tail comfortably hits the padding void ($x_1 = 0$). Circular = Linear!

- How much faster is this really?
- **Direct Linear Convolution:** Takes $O(L \cdot M)$ operations. If $L \approx M \approx N/2$, naive convolution takes roughly $O(N^2)$ steps.
- **Fast Convolution (using Radix-2 FFT):**
 - 2 Forward FFTs: $2 \times O(N \log N)$
 - 1 Element-wise multiplication: $O(N)$
 - 1 Inverse FFT: $O(N \log N)$

Total time: $O(N \log N)$ complex multiplications.

Real-world magnitude

Applying a 1-second reverb filter ($L = 44,100$) to 1-second of audio ($M = 44,100$):

- **Naive:** $L \times M \approx 1.94 \times 10^9$ operations.
- **Fast Convolution:** Padding to $N = 131,072 = 2^{17}$, taking $3 \times (N \log_2 N) \approx 6.6 \times 10^6$ operations.
- A massive speedup of purely algorithmic origin allowing real-time high-fidelity audio processing.

FFT for Prime Lengths

What if N is Prime? Bluestein's Algorithm

- We know Cooley-Tukey yields $O(N \log N)$ performance for powers of 2 (or composite $N = N_1 N_2$). But what if N is a large prime number (e.g., $N = 1,000,003$)?
- Then $N_1 = 1$, and Cooley-Tukey devolves right back to $O(N^2)$.
- **Bluestein's Algorithm** bounds any DFT constraint to $O(N \log N)$ by cleverly expressing the DFT as a linear convolution!

The Chirp-Z Identity

Start with the algebraic identity for powers:

$$(k - n)^2 = k^2 - 2kn + n^2 \implies kn = -\frac{(k - n)^2}{2} + \frac{k^2}{2} + \frac{n^2}{2}$$

Using this, we can substitute kn in the twiddle factor $W_N^{kn} = W_{2N}^{2kn}$ giving:

$$W_N^{kn} = W_{2N}^{-(k-n)^2} W_{2N}^{k^2} W_{2N}^{n^2}$$

- Applying the Chirp-Z Identity to the DFT definition:

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] W_N^{kn} \\ &= \sum_{n=0}^{N-1} x[n] \left(W_{2N}^{-(k-n)^2} W_{2N}^{k^2} W_{2N}^{n^2} \right) \\ &= W_{2N}^{k^2} \sum_{n=0}^{N-1} \underbrace{\left(x[n] W_{2N}^{n^2} \right)}_{a[n]} \underbrace{W_{2N}^{-(k-n)^2}}_{b[k-n]} \end{aligned}$$

- The summation is precisely a **linear convolution** evaluated at index k !

$$X[k] = W_{2N}^{k^2} \cdot (a * b)[k]$$

where $a[n] = x[n] W_{2N}^{n^2}$ for $n \in [0, N-1]$ and the chirp filter $b[n] = W_{2N}^{-n^2}$ for $n \in [-(N-1), N-1]$.

- We can evaluate this convolution using fast convolution!
- But in this case we need to handle the padding of $b[n]$ with care as n can be negative.

Visualizing Zero-Padding for Bluestein's Algorithm



- Linear convolution evaluates $b[k - n]$, so index ranges from $k - (N - 1)$ to k .
- Note that $k - n$ can be negative, so we need to handle negative indices when they wrap around at $k - n \pmod{M}$.
- We allocate the **positive indices** of $b[n]$ at the start of the size- M array, and the **negative indices** at the end, placing zeros in the middle to prevent overlapping.
- So, M must be at least $2N - 1$ to avoid overlapping.

- How does solving a multiplication via convolution help?
- Because we can process linear convolution utilizing **Fast Convolution with Zero-Padding!**
- Sequence $a[n]$ has length N . Sequence $b[n]$ is evaluated for $n \in -(N-1) \dots (N-1)$, meaning its support length is $2N-1$.
- We can pick any $M \geq 2N-1$ and zero-pad both sequences. We intentionally choose $M = 2^m$ so we can use the highly efficient Radix-2 DIT FFT!

Bluestein's FFT Algorithm

1. Form sequence $a[n]$ and periodized $b[n]$ to size $M = 2^{\lceil \log_2(2N-1) \rceil}$.
2. Compute $A = \text{FFT}(a, M)$ and $B = \text{FFT}(b, M)$ using Radix-2.
3. Compute $Y[k] = A[k]B[k]$.
4. Compute $y[n] = \text{IFFT}(Y, M)$.
5. Multiply by $W_{2N}^{k^2}$: $X[k] = y[k]W_{2N}^{k^2}$ for $k = 0 \dots N-1$.